

High-Performance Implementation of Multigrid Solver for Lattice QCD

张鑫

中国科学院近代物理研究所

2026 年 9 月 28 日

摘要

本文给出 PyQCU 面向 Wilson-Clover 传播子求解的高性能 MultiGrid 实现。方法以奇偶 Schur 算子为细层问题, 通过近零模、局部正交聚集和 Galerkin 投影生成递归粗层。Strict 路径采用 $D_l = X_l + H_l$, $\hat{D}_l = X_l^{-1} D_l$ 和 $D_{l+1} = R_l \hat{D}_l P_l$, 运行时保留完整粗层几何与 $X/Y/\hat{Y}$ 资产。CUDA-C++ 后端通过细层投影复用、粗层设备端标量与融合内核、CUDA Graph、分块/着色 Galerkin、持久工作区、校验缓存、rank-local halo 和非阻塞向量通信降低真实 wall time。性能评价严格采用 git tag test27: 66 个精确 combined 单元、132 条 side record, MG-2/3 在 35/44 个单元中优于 QUDA; 正式 trace-off MG 为 21/22。所有精确单元通过 full-operator 真残差门, 最大观测显存低于 85% 门限。此外, 本文给出 508 报告中的 64-1024 进程大格扩展数据, 作为 PyQCU 早期 DCU 路径的独立历史证据。

关键词: 格点 QCD; Wilson-Clover; MultiGrid; Bi-CGStab; FGMRES; CUDA; MPI

1 任务范围与证据口径

本文只讨论 PyQCU 的 MultiGrid 求解器及其 CUDA-C++ 实现。性能事实主要依据 docs/report_multigrid_quda_pyqcu_20260928.tex 与 git tag test27; 算子构造和实现机制同时参考 docs/report_pyqcu_mg_operator_construction_20260830.tex、docs/report_multigrid_optimized_20260927.tex 和当前源码。508 报告数据单列为“先前成果”, 不与 test27 的 Strict-MG 速度比混合。

2 传播子、Dslash 与奇偶消元

2.1 Wilson 与 Clover

规范链接 $U_{x,\mu} \in SU(3)$ 连接相邻格点。取 Wilson 参数 $r = 1$ ，裸跃迁核和 Wilson 算子为

$$(H\psi)_x = \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}} \right], \quad (2.1)$$

$$D_W = (m_0 + 4)I - \frac{1}{2}H = (m_0 + 4)(I - \kappa H), \quad \kappa = \frac{1}{2m_0 + 8}. \quad (2.2)$$

PyQCU 的求解器使用归一化形式 $D_{\text{pc}} = I - (\kappa/u_0)H$ ，当前 C++ 生产路径等价于 $u_0 = 1$ 。C++ 原始 Dslash 入口返回裸核 H ，不含单位项和 $-\kappa$ 因子。

Clover 项由四点规范环构造：

$$Q_{\mu\nu} = P_{\mu\nu} + P_{\nu,-\mu} + P_{-\mu,-\nu} + P_{-\nu,\mu}, \quad (2.3)$$

$$C_{\mu\nu} = Q_{\mu\nu} - Q_{\mu\nu}^\dagger, \quad (2.4)$$

$$T_p = -\frac{\kappa c_{\text{sw}}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu}, \quad A_p = I_p + T_p. \quad (2.5)$$

当前有效 $c_{\text{sw}} = 1$ 。Clover 项是 onsite 12×12 spin-color block，不扩大最近邻 stencil。

2.2 奇偶 Schur 消元

将场分为 even 和 odd 两个子空间，完整块算子为

$$D_{\text{pc}} = \begin{pmatrix} A_e & -\kappa H_{eo} \\ -\kappa H_{oe} & A_o \end{pmatrix}. \quad (2.6)$$

以奇数子格 Schur 补为例：

$$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}, \quad (2.7)$$

$$b_o^{\text{pc}} = b_o + \kappa H_{oe} A_e^{-1} b_e, \quad (2.8)$$

$$x_e = A_e^{-1} (b_e + \kappa H_{eo} x_o). \quad (2.9)$$

这是精确消元。外层 Krylov 迭代只作用于半体积未知量，最终仍使用完整 Wilson/Clover 算子的真残差判定收敛。

3 MultiGrid 算法

3.1 近零模、转移和 Galerkin 粗算子

设第 l 层算子为 A_l , 近零模满足

$$A_l B_l \approx 0. \quad (3.1)$$

在每个 aggregate 内对 B_l 做局部正交化得到延拓 P_l , 并令

$$R_l = P_l^\dagger, \quad P_l^\dagger P_l = I, \quad A_{l+1} = R_l A_l P_l. \quad (3.2)$$

$P_l P_l^\dagger$ 不是细层单位算子; 粗空间只消除 P_l 能表示的低模误差。代表性粗层维度若为 EN_c , 显式稠密存储需要 $O((EN_c)^2)$, 因此运行期只保存有限支撑 block/stencil。

3.2 预条件 Bi-CGStab

Algorithm 3.1. Preconditioned Bi-CGStab: solve $M^{-1}Ax = M^{-1}b$

Input : $A, b, x^{(0)}, M, \varepsilon$
 $r^{(0)} = b - Ax^{(0)}, \quad \tilde{r} = r^{(0)}$
for $i = 1, 2, \dots$
 $\rho_{i-1} = \tilde{r}^\dagger r^{(i-1)}$
 if $\rho_{i-1} = 0$ **then fail**
 if $i = 1$ **then** $p^{(1)} = r^{(0)}$
 else $\beta_{i-1} = \frac{\alpha_{i-1}\rho_{i-1}}{\omega_{i-1}\rho_{i-2}}; p^{(i)} = r^{(i-1)} + \beta_{i-1} (p^{(i-1)} - \omega_{i-1}v^{(i-1)})$
 solve $M\hat{p} = p^{(i)}; \quad v^{(i)} = A\hat{p}$
 $\alpha_i = \frac{\rho_{i-1}}{\tilde{r}^\dagger v^{(i)}}; s = r^{(i-1)} - \alpha_i v^{(i)}$
 if $\|s\|/\|b\| \leq \varepsilon$ **then** $x^{(i)} = x^{(i-1)} + \alpha_i \hat{p}$; **stop**
 solve $M\hat{s} = s; \quad t = A\hat{s}$
 $\omega_i = \frac{t^\dagger s}{t^\dagger t}$
 $r^{(i)} = s - \omega_i t; \quad x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}$
end for

3.3 PyQCU Strict 粗层分解

Strict 路径将粗层算子拆成

$$D_l = X_l + H_l, \quad \widehat{D}_l = X_l^{-1} D_l, \quad D_{l+1} = R_l \widehat{D}_l P_l. \quad (3.3)$$

轴链接资产满足

$$\widehat{Y}_{l,\mu}^f = X_l^{-1} Y_{l,\mu}^f, \quad \widehat{Y}_{l,\mu}^b = Y_{l,\mu}^b X_l^{-\dagger}. \quad (3.4)$$

细层是 compact target-parity 问题, 粗层保留 full geometry。运行时常规求解驻留 $X, X^{-1}, \widehat{Y}$; raw Y 只用于 setup 或诊断。

3.4 V-cycle 与右预条件 FGMRES

Algorithm 3.2. Recursive MG preconditioner and flexible right-preconditioned GMRES

Input : $A_l, r_l, \{S_{\text{pre}}, S_{\text{post}}, P_l, R_l\}_{\ell=0}^{L-1}$
if $l = L - 1$ **then** $z_l \leftarrow A_l^{-1} r_l$ (Bi-CGStab)
else
 $y_l \leftarrow S_{\text{pre}}^{\nu_1}(r_l)$
 $r_{l+1} \leftarrow R_l(r_l - A_l y_l)$
 $e_{l+1} \leftarrow \mathcal{M}_{l+1}^{-1} r_{l+1}$
 $z_l \leftarrow y_l + P_l e_{l+1}$
 $z_l \leftarrow S_{\text{post}}^{\nu_2}(z_l)$
end if

$r_0 = b - Ax^{(0)}, \quad v_1 = r_0 / \|r_0\|$
for $j = 1, \dots, m$
 $z_j = \mathcal{M}^{-1} v_j; \quad w = Az_j$
for $i = 1, \dots, j$
 $h_{ij} = v_i^\dagger w; \quad w = w - h_{ij} v_i$
end for
 $h_{j+1,j} = \|w\|; \quad v_{j+1} = w / h_{j+1,j}$
 apply Givens rotations to H_j
end for
 $x^{(k+1)} = x^{(k)} + Z_j y; \quad Z_j = [z_1, \dots, z_j]$
 recompute full true residual; restart if needed

4 CUDA-C++ 实现

4.1 细层与粗层瓶颈

Wilson Dslash 单次应用读取八个邻居。 $1 \pm \gamma_\mu$ 把四分量 spinor 投影为两个分量, 再执行 $SU(3)$ 矩阵乘法和累加。时空维尾置有利于合并访存。粗层相反, 向量较小但内核启动、标量复制、点积、全局归约和同步会主导 wall time。

CUDA-C++ 后端采用以下优化:

- 将 $\alpha, \beta, \omega, \rho$ 等标量保留在设备端 `device_vals`, 减少主机往返;
- 融合 $\beta/\rho, r/x$ 等元素更新, 减少内核数;
- 对固定粗层 Bi-CGStab 段使用 CUDA Graph, 降低 launch 开销;
- 对小最粗层使用 cooperative fused solve, 过大时安全回退;
- 使用持久 hierarchy、递归 arena 和外部 FGMRES workspace;
- runtime cache 记录 shape、dtype、identity 和逐 tensor SHA256。

4.2 Setup 与显存

Galerkin 构造按 source 着色并分批执行, 根据 workspace cap 自动选择 batch 大小。Strict 缓存使用 HDF5 保存 blocked basis、onsite inverse 和 $\hat{Y}$, 读取前校验每个 tensor, 写入使用临时文件和原子发布。若 setup 无法摊销到足够求解次数, MG 即使减少外层迭代也未必获得端到端加速。

4.3 分布式粗层通信

每个 rank 只持有本地粗块, halo 只交换活跃 face。c64 vector halo 使用

$$\text{active-face pack} \parallel \text{local base kernel} \rightarrow \text{nonblocking MPI} \parallel \text{local tail} \rightarrow \text{remote correction.} \quad (4.1)$$

complex64 默认启用 overlap; complex128 改变浮点累加顺序后曾由 32 次外层迭代退化到 451 次, 因此默认关闭, 只有显式开关并重新通过大格真残差门后才允许启用。当前 link halo 仍使用阻塞式 Sendrecv。

5 性能评价

5.1 验证协议

对照实现为 QUDA 1.1.0 MultiGrid; 设备为单卡 V100-SXM2-32GB 与双 P100-PCIE-16GB。矩阵覆盖 complex64/c128、BiCGStab reference、MG-2、MG-3、trace on/off 和三类格点。每个精确 side case 执行 1 cold、2 warmup、5 steady, 正式性能使用 trace-off median。

5.2 总体与分组结果

定义 $R = t_{\text{QUDA}}/t_{\text{PyQCU}}$ 。66 个精确单元整体中位 $R = 1.025$; MG-2/3 为 35/44 个单元有利, 正式 trace-off MG 为 21/22; BiCGStab reference 为 0/22。因此优势属于 MG 专项, 不能外推到全部求解器。

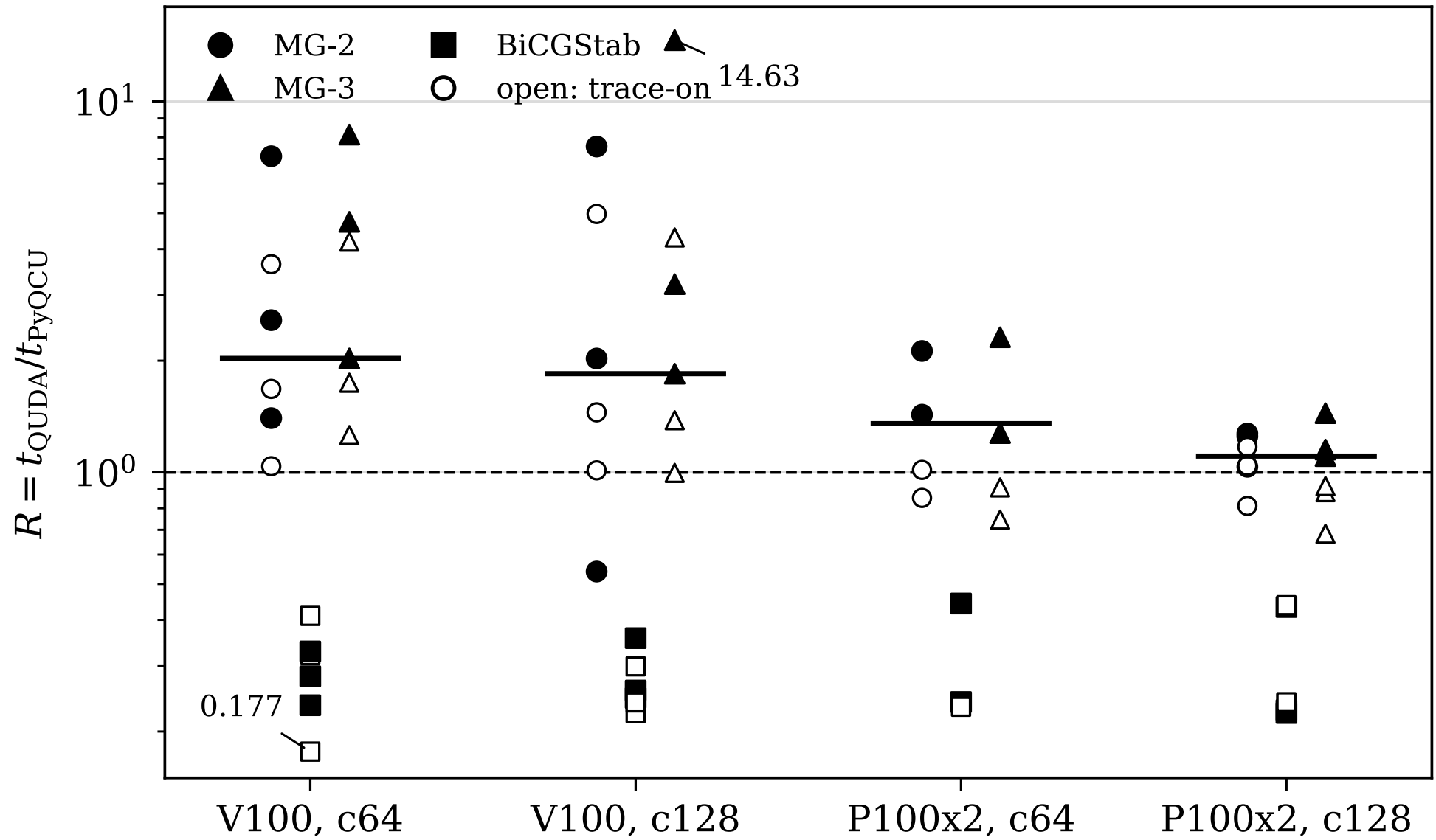


图 5.1: 66 个精确单元的逐点时间比。实心 symbol 为 trace-off, 空心为 trace-on; 水平短线为组内 trace-off 中位数。

表 5.1: test27 精确矩阵覆盖与公平性门禁

项目	观察值	要求
请求 combined 单元	72	计划矩阵
精确 combined 单元	66	66/66 通过
精确 side record	132	132/132 通过
1 cold / 2 warmup / 5 steady	132/132	全部完整
config/input hash 一致	66/66	全部一致
最大 full true residual	7.97×10^{-7}	$< 5 \times 10^{-6}$

表 5.2: trace-off 分组中位时间比 R

设备/精度	MG-2	MG-3	BiCGStab	样本
V100 / c64	2.5717	4.7366	0.2818	$n = 3$
V100 / c128	2.0269	3.2174	0.2580	$n = 3$
P100 $\times$ 2 / c64	1.7773	1.7944	0.3416	$n = 2$
P100 $\times$ 2 / c128	1.2502	1.1517	0.2284	$n = 3$

5.3 阶段与容量

trace-on 阶段份额中位数表明, PyQCU 的 coarse solve 与 other 合计约占 95%, QUDA 的主要成本集中在具名 coarse 阶段。两侧层级映射不同, 同名阶段不能直接跨库相加。

双 P100 c64 $16 \times 32 \times 32 \times 48$ 请求项三次尝试分别使用默认 overlap、PYQCU_MPI_OVERLAP=0 和 extended CPU staging, 峰值约 15.9、14.8、12.4 GB; 第三条路径仍超过 570 s setup 时限。对应 6 个 combined 项被显式替代为 16^4 , 不进入原大格速度比。

6 508 报告的大规模先前成果

张鑫 508 报告记录 4 卡 Pre-Wukong DCU 环境中的传播子大格测试。报告中的强扩展序列由 64 扩展到 1024 进程/线程; 在 1024 规模, Wilson 与 Clover 的累计报告加速比分别为 3.12 和 7.19。报告估算 64 卡池峰值约为 2048 GiB。右图给出 Clover 与 Wilson 单次迭代耗时差随等效单进程体积增长的数据。

该数据用于证明早期 DCU/CUDA 路径已具备正确性和大规模扩展趋势。报告同时指出, 当时 MultiGrid 仍属于前期验证版本, 因此不能替代 test27 的 Strict-MG 正式对照。

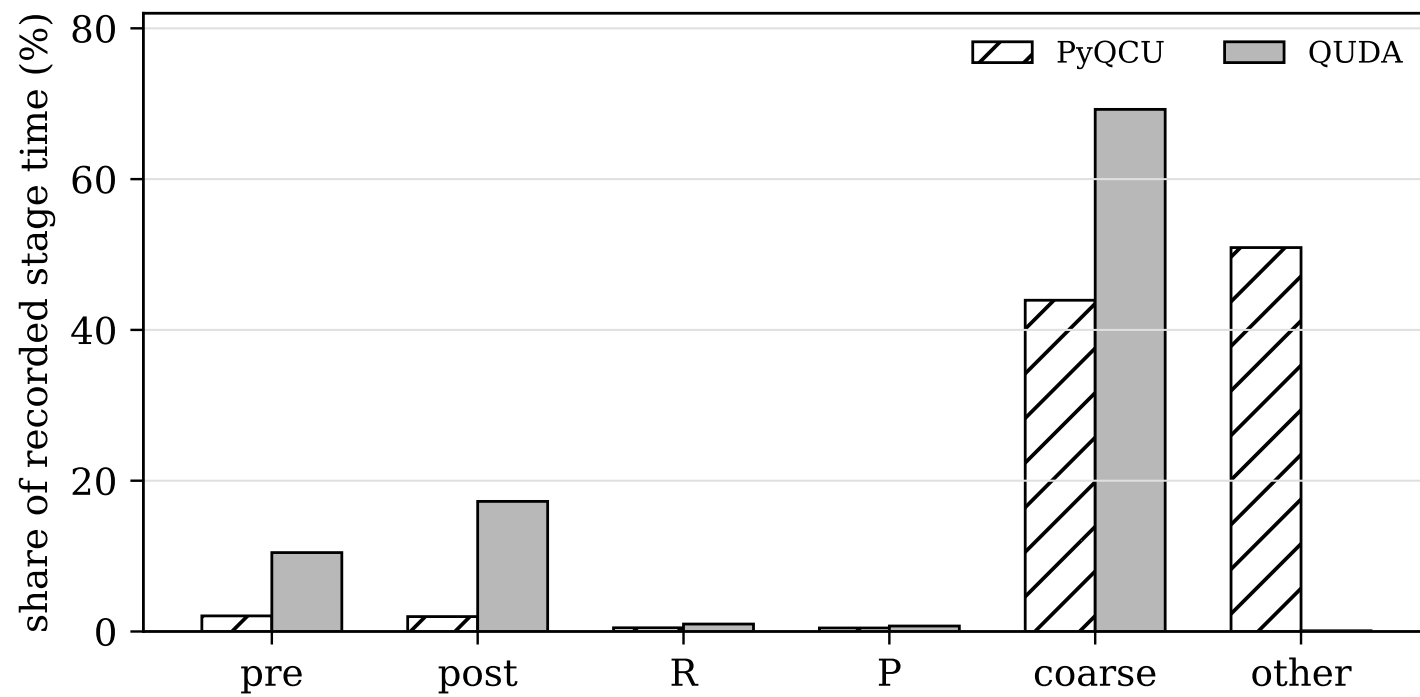


图 5.2: recorded stage time 的中位数份额。

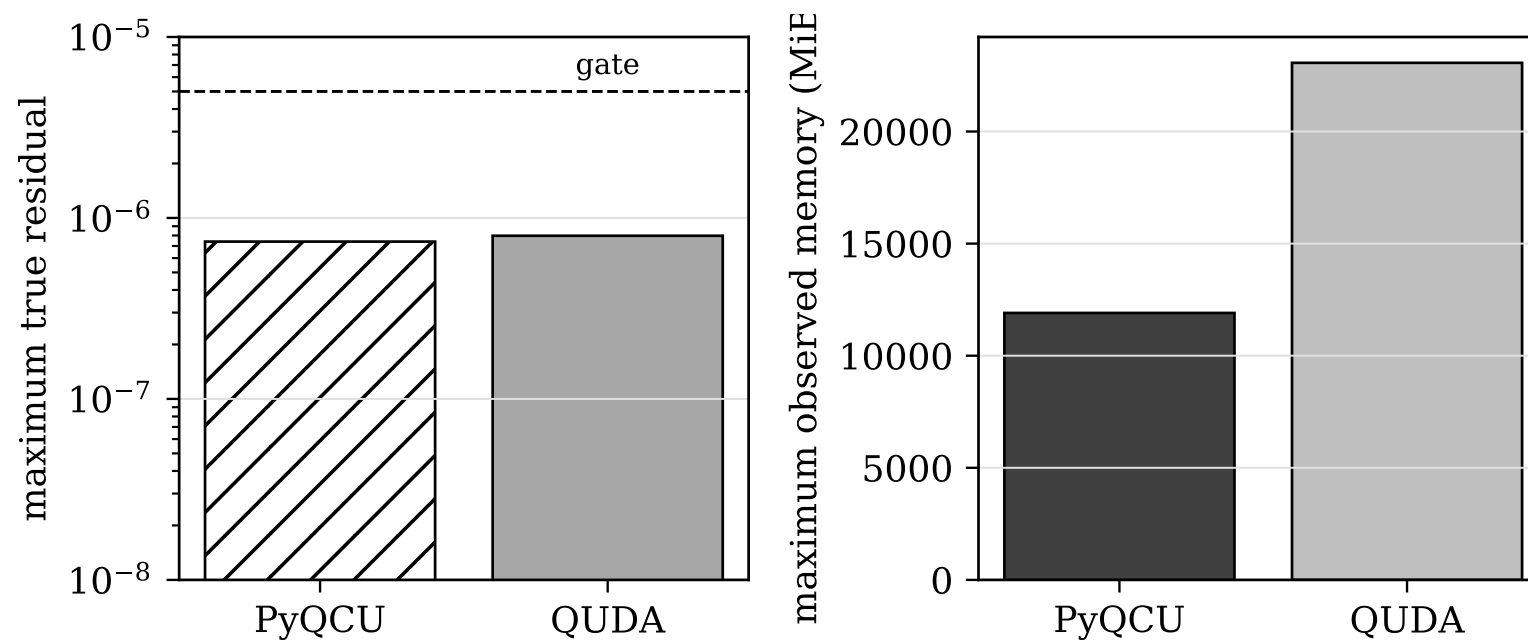


图 5.3: 最大 full true residual 与 device-wide 最大观测显存。

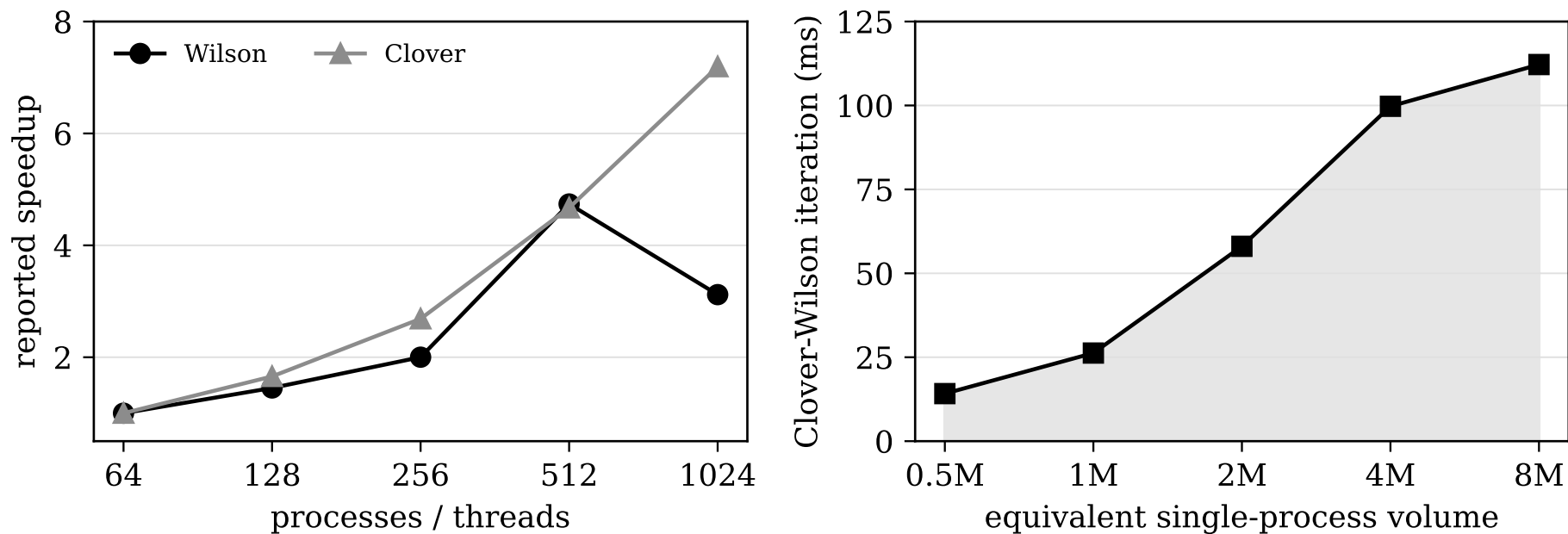


图 6.1: 508 报告中的 64–1024 强扩展曲线与弱扩展差异曲线。

PyQCU: Mass为0.05, tol(x_o)为1e-12, 数据类型为complex128, _BLOCK_SIZE_=256, 使用节点绑定脚本, 使用Wilson与Clover算子时, 用面源求解传播子的DCU性能测试, 其中 (X:128,Y:128,Z:128,T:256) (具体数据参考<https://gitee.com/zhangxin8069/PyQCU/tree/test9>)

序号	线程数	切分方式	等效单卡格点数量	WilsonBis tabCg迭代数	WilsonBis tabCg最终残差范数的平方 (x_e)	WilsonBis tabCg耗时/s	WilsonBis tabCg单次迭代耗时/ms	WilsonBis tabCg加速比	CloverBis tabCg迭代数	CloverBis tabCg最终残差范数的平方 (x_e)	CloverBis tabCg耗时/s	CloverBis tabCg单次迭代耗时/ms	CloverBis tabCg加速比
1	128	2,4,4,4	4194304	162	4.36466E-23	9.078154613	56.03799144	1.000	170	6.14E-23	28.5984921	168.2264241	1.000
2	256	4,4,4,4	2097152	156	9.40601E-23	6.953258005	44.5721667	1.257	159	6.36E-23	16.31604647	102.6166444	1.639
3	512	4,4,4,8	1048576	157	2.17205E-23	5.462116936	34.79055373	1.611	157	5.33213E-23	9.582966548	61.03800349	2.756
4	1024	4,4,8,8	524288	157	1.14308E-23	3.979191492	25.34516874	2.211	149	3.79539E-23	5.88710933	39.51080087	4.258

显存峰值占用大概为64GBi*64卡*50%*2=4096GBi

图 6.2: 508 报告 TEST9 原始强扩展数据表。

7 讨论与结论

1. V100 上 MG 优势最明显，c64/c128 MG-3 trace-off 中位数分别为 4.74 和 3.22。
2. 双 P100 在 trace-off 下总体有利，但 MG-2/3 中位数仅为 1.15–1.79，且大格容量受显存与 setup 时限限制。
3. BiCGStab reference 为 0/22，说明优势来自 MG，不是全部求解器。
4. 最大真残差满足门限；PyQCU 最大观测显存约为 QUDA 的一半。
5. 508 报告证明早期 DCU 大规模扩展能力，但不能替代当前 Strict-MG 的正式性能边界。

下一阶段应聚焦 coarse solve、残差检查、全局归约和 setup 显存，继续推进 device-aware MPI、持久 request、混合精度策略与国产加速器端到端验证。

A 复现入口

```
source ./env.sh
bash ./build.sh
bash ./install.sh
```

```
python -B pyqcu/testing/qcu/strict/quda_comparison/run_strict_fast.py \
    --fail-fast --json strict-fast.json
```

```
mpirun -np 2 python -B \
    pyqcu/testing/qcu/strict/quda_comparison/strict_mpi_solve_probe.py \
    --shape 8 8 8 16 --grid 2 1 1 1 --dtype c64 \
    --galerkin-mode colored
```

B 原始证据

表 B.1: test27 最终证据文件

文件	内容
units.csv	66 个精确单元的总时间、迭代数、真残差和显存
stages.csv	逐层 pre/post/R/P/coarse/other 阶段记录

文件	内容
references.csv	BiCGStab cold/warmup/steady 原始样本
requested_matrix_status.csv	72 个请求单元状态
capacity_substitutions.csv	6 个大格容量替代项
summary_analysis.json	覆盖、分组和残差汇总

参考文献

- [1] K. G. Wilson, Confinement of quarks, *Physical Review D* **10**, 2445 (1974).
- [2] B. Sheikholeslami and R. Wohlert, Improved continuum limit lattice action for QCD with Wilson fermions, *Nuclear Physics B* **259**, 572 (1985).
- [3] R. Babich et al., Adaptive multigrid algorithm for the lattice Wilson–Dirac operator, *Physical Review Letters* **105**, 201602 (2010).
- [4] QUDA Collaboration, QUDA: A library for lattice QCD computations on GPUs. <https://github.com/lattice/quda>
- [5] M. Rottmann et al., DD- α AMG: Adaptive algebraic multigrid for lattice QCD. <https://github.com/mrottmann/DDalphaAMG>
- [6] E. Weinberg, quantum-mg: Multigrid solver package for lattice QCD. <https://github.com/weinbe2/quantum-mg>
- [7] 张鑫, PyQCU: Python/Cython and CUDA–C++ lattice QCD toolkit. <https://github.com/zhangxin8069/PyQCU>